



13. Graphs

Connecting Data Beyond Hierarchies

Previous Sections:

 [12. Priority Queues](#)

 [12.1. Heaps](#)

Contents:

[1. What is a Graph?](#)

[1.1. Relationship Between Trees and Graphs](#)

[2. Types of Graphs](#)

[2.1. Null Graph and Trivial Graph](#)

[2.2. Undirected and Directed Graphs](#)

[2.3. Connected and Disconnected Graphs](#)

[2.4. Regular and Complete Graphs](#)

[2.5. Cycle Graph and Cyclic Graph](#)

[2.6. Weighted Graphs](#)

[2.7. Bipartite Graphs](#)

[3. Applications of Graphs](#)

[3.1. Computer Science](#)

[3.2. Friend Suggestion in Social Media](#)

[3.3. Social Media Analysis](#)

[3.4. Map and Navigation Applications](#)

[3.5. World Wide Web](#)

[3.6. Path-Finding Problems](#)

[3.7. Operating Systems](#)

[3.8. Network Monitoring](#)

[3.9. Financial Trading](#)

[3.10. Internet of Things \(IoT\)](#)

[3.11. Autonomous Vehicles](#)

[3.12. Disease Surveillance](#)

[Advantages of Graphs](#)

[Disadvantages of Graphs](#)

[Summary](#)

[References](#)

Up until now, most non-linear data structures we have studied, such as Trees and Heaps, follow a hierarchical structure. Every node has a clear parent-child relationship, and traversal usually begins from a single root.

But what happens when data does not naturally fit into a hierarchy?

Consider:

- Social networks where users are connected to many other users.
- Road maps where cities are connected through multiple routes.
- Computer networks where devices communicate with one another.
- Flight routes connecting airports worldwide.

These structures are far more complex than trees because connections can occur between any pair of nodes. They may even form loops that allow us to return to where we started. This is where **Graphs** become important.

Graphs are among the most fundamental concepts in Computer Science. They are also heavily studied in Discrete Mathematics under a branch known as **Graph Theory**.

At first glance, Graph Theory and Graph Data Structures may appear identical. In reality, they focus on different perspectives of the same concept.

Graph Theory	Graph Data Structure
Focuses on mathematical properties of graphs	Focuses on representing graphs in computer systems

Graph Theory	Graph Data Structure
Studies proofs, theorems, and graph characteristics	Studies storage, traversal, and algorithms
Concerned with abstract relationships	Concerned with practical implementation

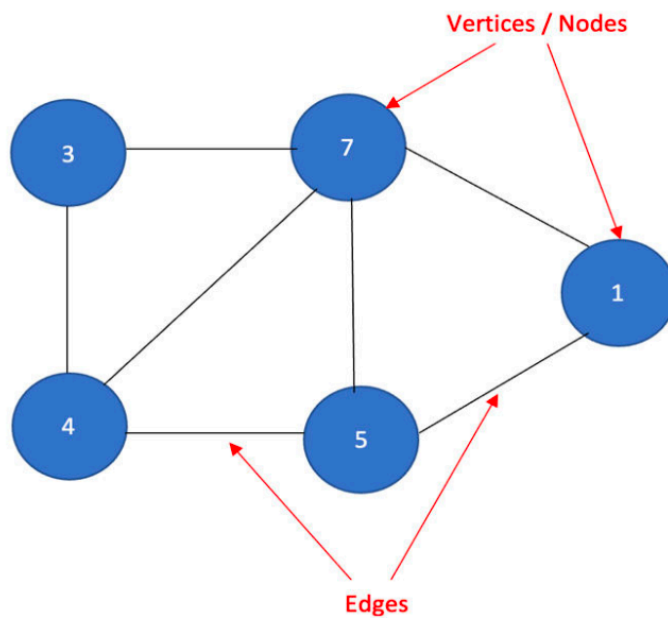
Although their goals differ, both fields study the same underlying structure and complement one another.

In this course, we focus on graphs as a **data structure**, examining how they are represented, categorized, and utilized within computer systems.

1. What is a Graph?

A **Graph** is a non-linear data structure consisting of two fundamental components:

- **Vertices (Nodes)** – represent entities.
- **Edges (Arcs)** – represent connections between entities.



A graph is typically represented as: $G(V, E)$. Where:

- V = Set of vertices
- E = Set of edges

For example, using the graph above:

- Vertices: 1, 3, 4, 5, 7
- Edges: (1, 5); (1, 7); (5, 7); (5, 4); (7, 4); (7, 3); (3, 4), ...

Unlike Trees, Graphs may contain cycles. In tree, there is only one path from the root to any node. In Graph, visiting the end of it and we eventually return to the starting point. Such loop is known as a cycle.

1.1. Relationship Between Trees and Graphs

A Tree can actually be viewed as a special type of Graph.

- Graph
 - Can contain cycles
 - Can contain multiple paths
 - May be disconnected
- Tree
 - No cycles
 - Exactly one path between nodes
 - Always connected

Therefore: ***Every Tree is a Graph, but not every Graph is a Tree.***

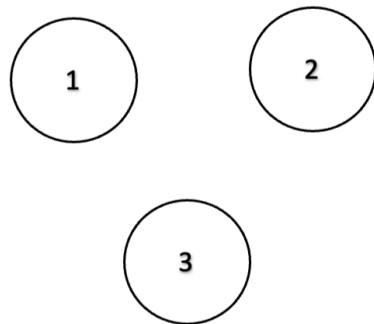
2. Types of Graphs

Graphs can be classified into many categories depending on how vertices and edges are organized.

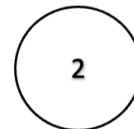
2.1. Null Graph and Trivial Graph

These are the simplest forms of graphs.

- **Null Graph:** A graph containing vertices but no edges. No vertices are connected.
- **Trivial Graph:** A graph containing only one vertex. This is the smallest graph possible.



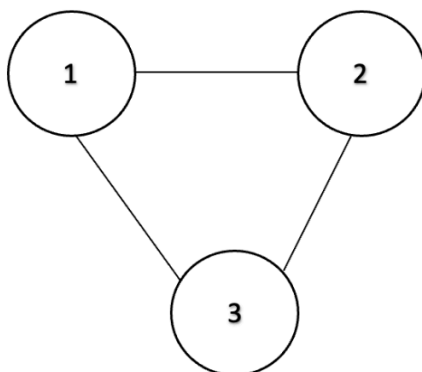
Null Graph



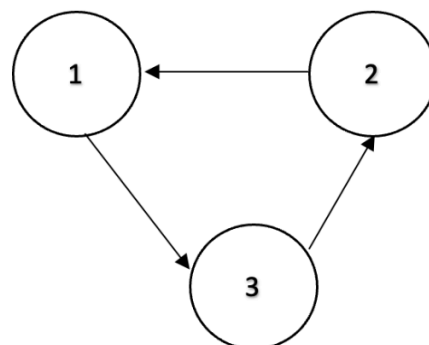
Trivial Graph

2.2. Undirected and Directed Graphs

- **Undirected Graph:** The edges have no direction. Travel is possible in both directions: $A \leftrightarrow B$. Mathematically: $(A, B) = (B, A)$. The order does not matter.
- **Directed Graph (Digraph):** The edges contain directions. Movement is only allowed from A to B. Mathematically: $(A, B) \neq (B, A)$. The order matters.



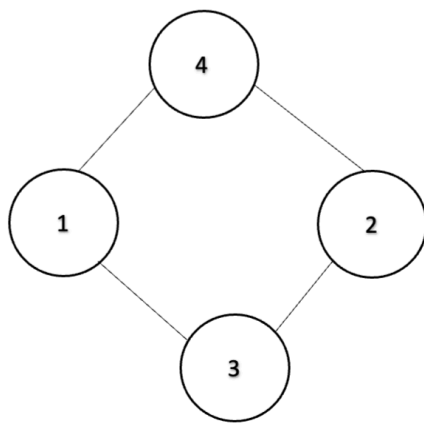
Undirected Graph



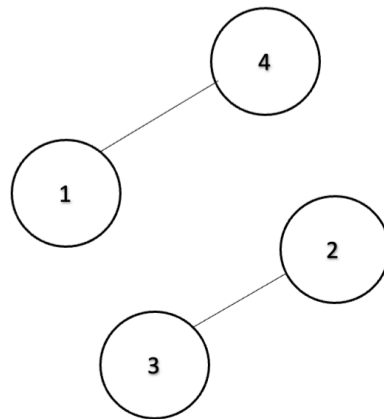
Directed Graph

2.3. Connected and Disconnected Graphs

- **Connected Graph:** Every vertex can be reached from every other vertex. Any node can eventually reach any other node.
- **Disconnected Graph:** At least one vertex cannot be reached from another. The graph consists of separate components.



Connected Graph



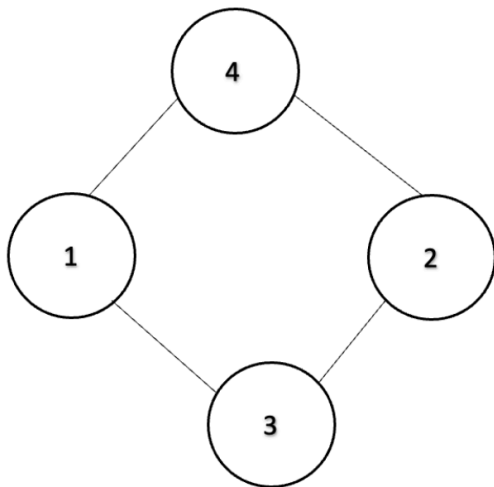
Disconnected Graph

- **Connected Components:** A disconnected graph may be divided into smaller connected graphs. Each component is internally connected.

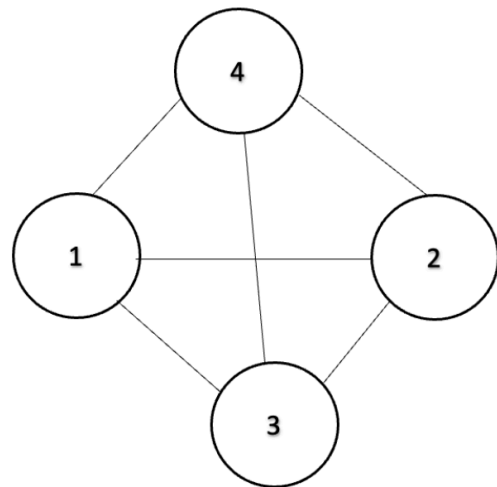
2.4. Regular and Complete Graphs

- **Degree of a Vertex:** The **degree** of a vertex is the number of edges connected to it.
- **Regular Graph:** A graph is called **K-Regular** if every vertex has the same degree K.
- **Complete Graph:** A graph is complete if every pair of vertices is connected.
 - For a complete graph with n vertices: $E = \frac{n(n-1)}{2}$
 - Example: $n = 4 \rightarrow E = \frac{4(4-1)}{2} = 6$

- Degree of each vertex: $\deg(v) = n - 1$. Therefore: Complete Graph (n vertices) $\rightarrow (n - 1)$ -Regular Graph.
- ***Every complete graph is also a regular graph.***



2-Regular Graph

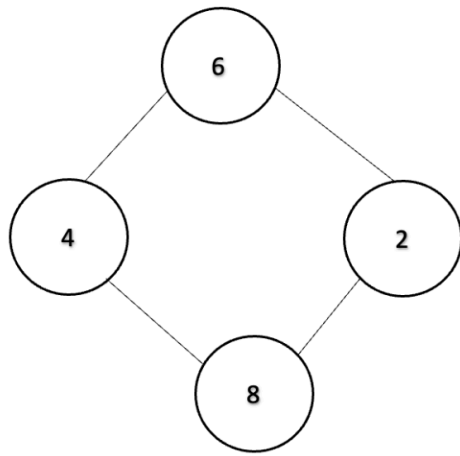


Complete Graph

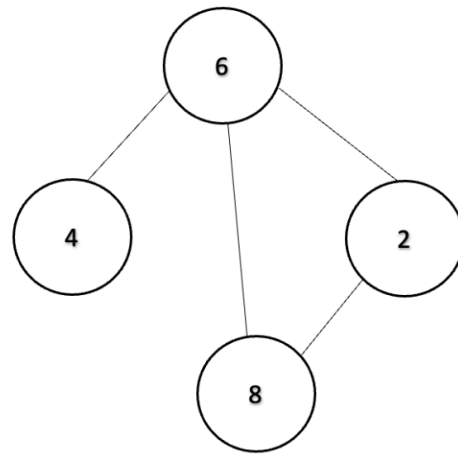
2.5. Cycle Graph and Cyclic Graph

These two terms are often confused.

- **Cycle Graph:** A graph whose vertices form exactly one cycle. Properties:
 - Every vertex has degree 2.
 - Forms a closed loop.
 - No extra edges.
- **Cyclic Graph:** A graph containing at least one cycle.
 - Cycle: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$
 - Since at least one cycle exists, the graph is cyclic.



Cycle Graph



Cyclic Graph

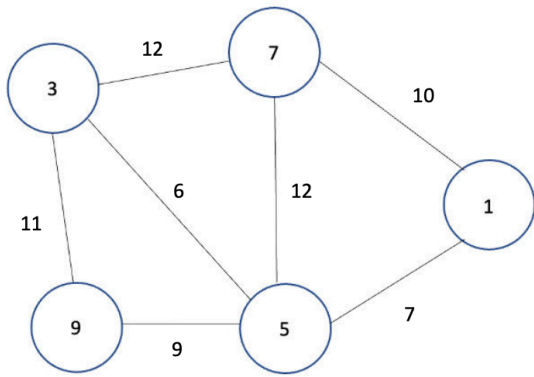
- Comparison
 - Cycle Graph → Exactly one cycle structure
 - Cyclic Graph → Contains one or more cycles

Every cycle graph is cyclic, but not every cyclic graph is a cycle graph.

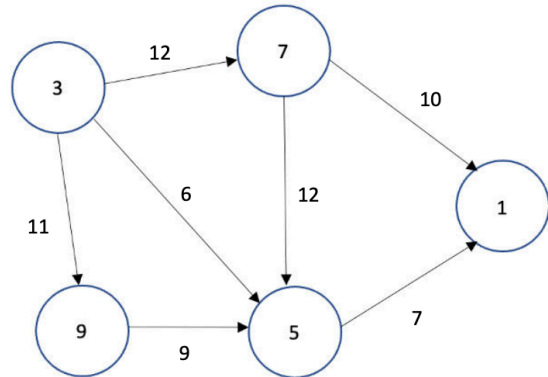
2.6. Weighted Graphs

In many real-world applications, edges carry additional information. Examples: Distance; Cost; Time; Traffic; Bandwidth. These values are called **weights**.

- Weighted Graph: Graph with values assigned for the edges.
- Directed Weighted Graph: Both direction and weight are considered.



Undirected Weighted Graph



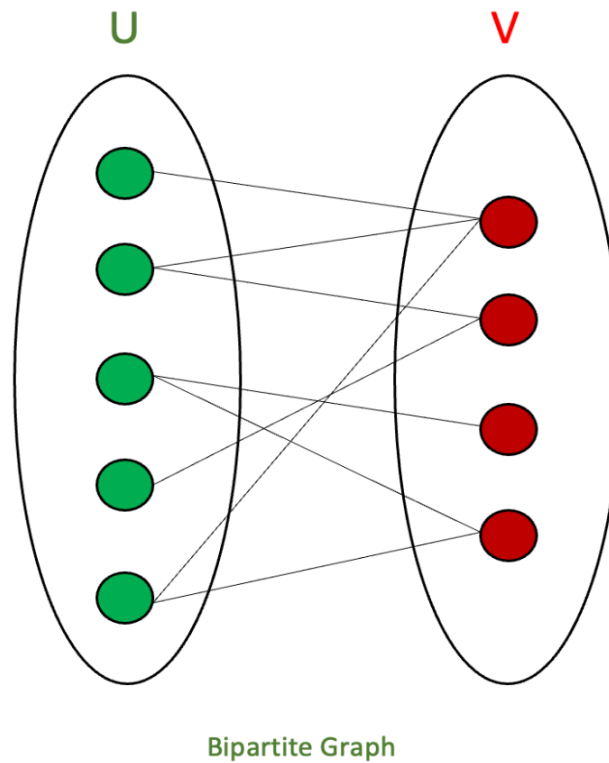
Directed Weighted Graph

- Weighted graphs are heavily used in:
 - GPS navigation
 - Network routing
 - Airline scheduling
 - Path-finding algorithms like Dijkstra's Algorithm; Bellman-Ford Algorithm; A* Search Algorithm

2.7. Bipartite Graphs

A graph is called **Bipartite** if its vertices can be divided into two separate sets such that no vertices within the same set are connected.

In other words, all connections occur between the two sets.



Real-World Examples

- Students and Courses

Students		Courses
Alice	-----	Programming
Bob	-----	Database
Charlie	-----	Programming

- Customers and Products

Customers		Products
A	-----	Laptop
B	-----	Mouse
C	-----	Laptop

- Movie Recommendation Systems

Users ↔ Movies

- Social Matching Systems

Employers ↔ Candidates

Bipartite graphs are extensively used in: Recommendation systems; Matching problems; Scheduling systems; Resource allocation algorithms

3. Applications of Graphs

Graphs are one of the most versatile data structures in Computer Science because they can naturally represent relationships and connections between objects. Whenever entities are connected in some way, graphs can often be used to model the problem efficiently.

In modern computing, graphs appear everywhere—from social networks and navigation systems to operating systems and artificial intelligence.

3.1. Computer Science

Graphs are widely used throughout Computer Science to represent the flow of computation, dependencies between tasks, and relationships among data.

Examples include:

- Compiler optimization
- Dependency management
- Database systems
- Artificial Intelligence
- Operating Systems

The graph below shows how different computations depend on one another.

```

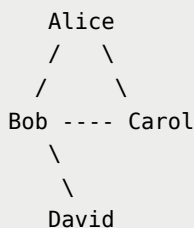
Task A
  |
  v
Task B -----> Task D
  |

```

3.2. Friend Suggestion in Social Media

Social media platforms such as Facebook, Instagram, and LinkedIn use graphs to represent relationships between users.

- Users are represented as vertices.
- Friendships are represented as edges.

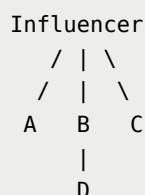


Suppose Alice and David are not connected. Since both share Bob as a mutual friend, the system may suggest friend: **Alice ↔ David**

This is one of the most common real-world applications of undirected graphs.

3.3. Social Media Analysis

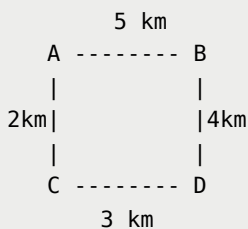
Modern social media platforms generate enormous amounts of data every second. Graphs help analysts identify: Trending topics; Communities; Influencers; User behavior; Sentiment propagation



The central node has the most influence because many users are connected to it.

3.4. Map and Navigation Applications

Navigation systems such as: Google Maps; Apple Maps; Waze use weighted graphs to determine optimal routes.



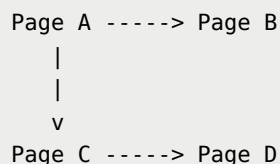
Vertices represent locations. Edges represent roads. Weights represent distance, or travel time, or fuel cost.

Graph algorithms can determine the shortest or fastest route.

3.5. World Wide Web

The Internet itself can be modeled as a giant graph.

- Web pages are vertices.
- Hyperlinks are directed edges.

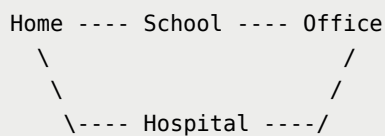


Since hyperlinks have directions, the Web is typically represented using directed graphs. Search engines analyze these connections to rank pages and determine importance.

3.6. Path-Finding Problems

Graphs are extensively used to solve path-finding and optimization problems. Examples include:

- Finding the shortest route between two cities.
- Determining the cheapest delivery route.
- Robot navigation.
- GPS systems.

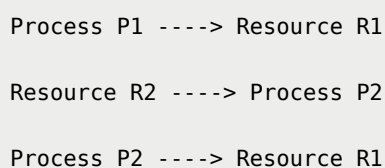


Graph algorithms determine the most efficient path between locations.

Common algorithms include: Dijkstra's Algorithm; Bellman-Ford Algorithm; Floyd-Warshall Algorithm; A* Search Algorithm

3.7. Operating Systems

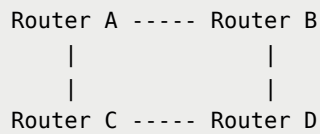
Operating systems use graphs to manage resource allocation. A common model is the **Resource Allocation Graph (RAG)**.



If a cycle forms: $P1 \rightarrow R1 \rightarrow P2 \rightarrow R2 \rightarrow P1$, a **deadlock** may occur. This means processes are waiting indefinitely for resources held by one another.

3.8. Network Monitoring

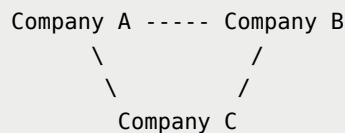
Computer networks can be represented as graphs. Routers are vertices. Connections are edges.



Network administrators use graph algorithms to detect bottlenecks, monitor traffic, identify failures, improve routing efficiency.

3.9. Financial Trading

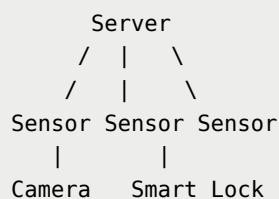
Financial institutions use graphs to analyze relationships between stocks, companies, markets, transactions



Graphs help identify market trends, trading opportunities, fraudulent activities, correlated assets. This is particularly important in high-frequency trading systems.

3.10. Internet of Things (IoT)

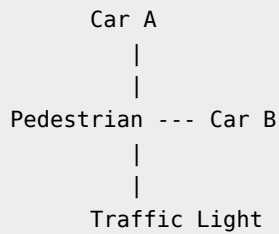
IoT systems contain thousands or even millions of interconnected devices.



Graphs help manage device communication, detect anomalies, optimize performance, analyze sensor data

3.11. Autonomous Vehicles

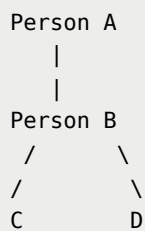
Self-driving vehicles constantly build graphs of their surroundings.



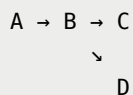
Graph algorithms allow vehicles to plan routes, avoid obstacles, predict movements, navigate safely.

3.12. Disease Surveillance

Graphs are frequently used in epidemiology to model disease transmission.



If Person A becomes infected:



Health organizations can use graph analysis to track outbreaks, predict spread patterns, identify high-risk individuals, implement containment strategies.

This became particularly important during global pandemics.

Advantages of Graphs

Graphs offer many benefits when representing complex relationships.

- **Effective Representation of Complex Data:** Graphs excel at modeling relationships that are difficult to represent using linear structures. Such as:
 - People ↔ Friends
 - Cities ↔ Roads
 - Webpages ↔ Hyperlinks
- **Efficient Processing with Graph Algorithms:** Specialized algorithms allow graphs to be processed efficiently. Examples are Breadth-First Search (BFS); Depth-First Search (DFS); Dijkstra's Algorithm; Minimum Spanning Tree Algorithms
- **Powerful Network Analysis:** Graphs are ideal for studying networks. Applications include:
 - Social networks
 - Business relationships
 - Organizational structures
 - Communication systems
- **Excellent for Path-Finding:** Graphs naturally model route optimization problems (**Source** → **Destination**). Algorithms can determine shortest path, fastest path, or the cheapest path
- **Highly Visual:** Graphs provide intuitive visual representations of relationships. Complex interactions become easier to understand.
- **Useful in Machine Learning:** Graphs are increasingly important in modern AI systems. Examples include Recommendation systems; Fraud detection; Knowledge graphs; Graph Neural Networks (GNNs)

Disadvantages of Graphs

Despite their advantages, graphs also have limitations.

- **Limited Attribute Representation:** Graphs primarily focus on relationships. Additional structures are often needed to store detailed properties of vertices and edges.

- **Difficult to Interpret:** Large graphs can become extremely complex. Understanding large-scale graphs may require advanced visualization techniques.
- **High Computational Cost:** As the number of vertices and edges increases. Memory usage increases. Processing time increases. Algorithms become more expensive. For very large networks, specialized graph databases may be required.
- **Sensitive to Data Quality:** A graph is only as accurate as its underlying data. Problems include missing data, incorrect relationships, and duplicate connections. Poor-quality data can lead to misleading conclusions.
- **Many Graph Variants:** Different graph types are designed for different purposes like Directed Graphs; Undirected Graphs; Weighted Graphs; Bipartite Graphs; Complete Graphs. Selecting the appropriate graph type can sometimes be challenging.
- **Privacy Concerns:** Graphs may reveal sensitive information about individuals and organizations. Examples are social connections, communication patterns, and financial relationships. Therefore, graph analysis often requires careful consideration of privacy and security issues.

Summary

Graphs are among the most powerful and flexible data structures in Computer Science. Unlike linear structures and trees, graphs are capable of representing complex relationships among entities through vertices and edges.

Their applications extend far beyond theoretical studies, appearing in social networks, navigation systems, operating systems, web search engines, network management, financial analysis, Internet of Things (IoT) systems, autonomous vehicles, and disease surveillance. Whenever relationships, connectivity, or path-finding are involved, graphs often provide the most natural solution.

Graphs offer numerous advantages, including their ability to model complex networks, support efficient graph algorithms, solve shortest-path problems, and provide intuitive visual representations. These strengths make them indispensable in modern computing and data analysis.

However, graphs also present challenges. Large graphs may be difficult to interpret, computationally expensive to process, and heavily dependent on data quality. Furthermore, graph analysis can introduce privacy concerns when sensitive relationships are involved.

Despite these limitations, graphs remain one of the most important data structures in Computer Science because they provide a practical way to model and analyze the interconnected world around us.

In the next section, we will learn how graphs are stored inside a computer using **Adjacency Matrices** and **Adjacency Lists**, two of the most common graph representation techniques.

References

https://drive.google.com/file/d/1R4R6eGz_XZEva12XTJEv3oIhZt1aet4x/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/graph-data-structure/>

<https://www.programiz.com/dsa/graph>

Next Section:

 [13.1. Representation of Graphs](#)